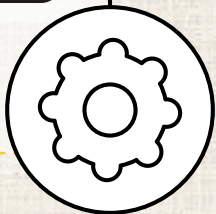


02 강

파이썬 시작!

첨단공학부
정재화 교수





1. 파이썬의 시작

- ✓ 파이썬의 기원
- ✓ Pythonic

2. 파이썬 개발 환경

- ✓ CPython
- ✓ 개발 환경

3. 파이썬 기초 문법

- ✓ 데이터 타입
- ✓ 함수



학습하기

1

파이썬의 시작

파이썬의 기원

■ 히도 판 로섬(Guido van Rossum) 1991년 개발



- ✓ 네덜란드 암스테르담 대학에서 컴퓨터 전공
- ✓ 크리스마스 휴가 기간 프로그래밍 프로젝트에서 시작
- ✓ 코미디 'Monty Python's Flying Circus'를 따라 명명

■ 파일 관리, 프로그램 실행, 반복 작업 자동화 등의 기능을 수행하는 언어 체계 용도로 개발

■ 프로그래밍을 더욱 인간 친화적이고 접근 가능한 분야로 변화시킨 혁신가로 평가

파이썬의 기본 철학

■ 읽기 쉽고 간결한 문법을 목표로 설계

- ✓ 프로그래밍 패러다임, 문법에 따라 다양한 표현 가능
- ✓ 다중 프로그래밍 패러다임 채용
 - 알고리즘을 프로그래밍 언어로 표현하는 접근 방식
 - 명령형 프로그래밍, 절차적 프로그래밍, 객체지향 프로그래밍, 함수형 프로그래밍 패러다임 등

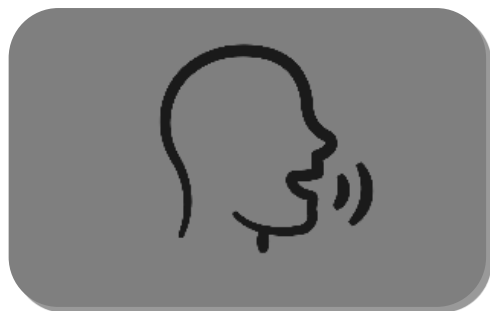
■ 다목적 활용

- ✓ 응용 프로그램과 웹, 백 엔드 개발, IoT, 데이터 과학, 인공지능&딥러닝 분야 뿐만 아니라 교육적인 목적으로도 활용

- 단순히 파이썬으로 작성된 코드가 아닌,
명료하고 아름답게 문제를 해결하는
사고방식을 의미



독립적



인간적



직관적



효율적



개방적

플랫폼 독립적

■ Write Once, Run Anywhere

- ✓ 하드웨어나 시스템 구조에 비의존적
- ✓ Windows, macOS, Linux 등 운영체제(플랫폼)에 관계없이 동일한 코드 실행이 가능
 - 인터프리터가 각 OS에 맞게 코드 해석

■ 표준 라이브러리 & 가상환경 지원

- ✓ 다양한 플랫폼에서 동일 기능 수행
- ✓ 개발 환경을 손쉽게 복제·이식 가능

I 인간적 & 직관적

■ 인간 중심의 철학

- ✓ 명확함과 가독성을 가장 중요시
- ✓ 초보자에게 친숙하고 전문가에게도 강력한 언어

■ 실행할 수 있는 의사 코드(executable pseudocode) 수준의 문법

- ✓ 영어 문장처럼 읽히는 코드 구조
- ✓ 불필요한 기호 없이 의미 중심의 구문 설계

```
1 if 3 in [1,3,5,7]: print("3이 들어있습니다")
```


효율적

- 간결하지만 강력한 표현력
- 풍부한 표준 라이브러리

```
1 int i, n;  
2 int sum = 0  
3  
4 printf("입력:");  
5 scanf("%d", &n);  
6  
7 for(i = 0; i < n; i++)  
8 {  
9     sum += i;  
10 }  
11  
12 print("합은 %d", sum);
```

C

```
1 n = int(input("입력:"))  
2 sum = 0  
3  
4 for i in range(1, n+1):  
5     sum += i  
6  
7 print(f"합은 {sum}")
```

Python

개방적

■ 많은 개발자의 의견을 수용하고 토론하며 발전하는 언어

- ✓ 오픈소스 생태계를 통한 공유·재사용·확장성 강화
- ✓ 풍부한 커뮤니티 자료로 문제 해결 속도 향상

■ 대형 커뮤니티를 기반으로 선순환 구조의 생태계 구축

- ✓ 새로운 기능, 라이브러리를 지속적으로 개발·공유
- ✓ 공식 문서, 포럼(Stack Overflow, Reddit 등), GitHub 등을 통해 문제 해결과 학습 자료를 공유
- ✓ 수십만 개의 라이브러리가 거의 모든 분야를 지원

학습하기

2

파이썬 개발 환경

파이썬의 언어적 특징

■ 플랫폼 독립적이며 인터프리터식 객체지향적, 동적 타이핑(dynamically typed) 대화형 언어

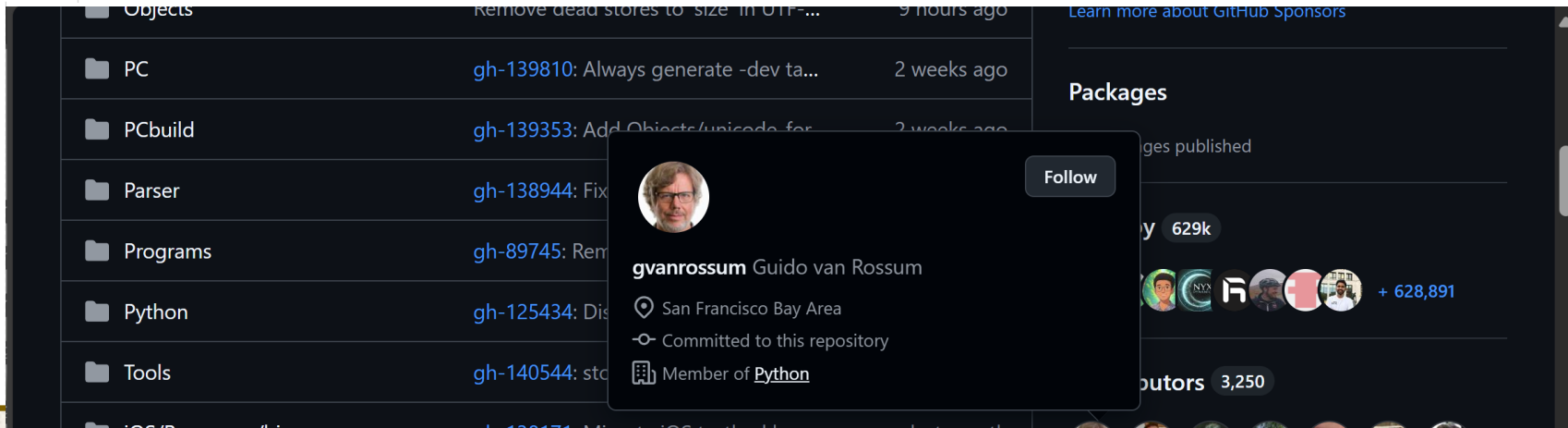
- ✓ 윈도우, 리눅스, 유닉스, 맥OS 등 다양한 운영체제(플랫폼)에서 별도의 컴파일 없이 실행 가능
- ✓ CPython, PyPy, Cython, Jython 등 다양한 인터프리터 환경 사용 가능
- ✓ 프로그램을 객체로 모델링
- ✓ 변수의 자료형을 지정하지 않음
- ✓ 작성한 코드에 대한 수행 결과를 바로 확인하고 디버깅하면서 코드 작성 가능

■ C 언어로 개발된 파이썬 인터프리터

- ✓ 파이썬 표준 라이브러리, 대부분의 패키지들이 CPython을 기준으로 개발·테스트됨
- ✓ C 구현 라이브러리와 연동을 통한 확장에 최적

■ 오픈소스 커뮤니티의 기여로 지속적 발전

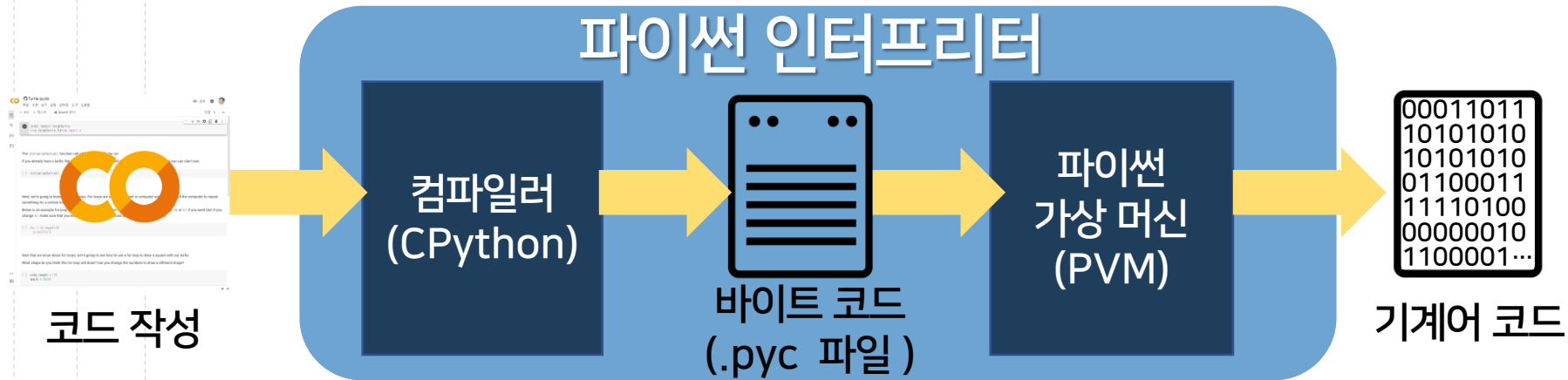
- ✓ <https://github.com/python/cpython>



파이썬 프로그램 실행과정

파이썬 애플리케이션은 소스 코드 형태로 배포

- ✓ CPython이 컴파일 후 바이트코드 .pyc 파일 생성
- ✓ 파이썬 가상머신은 바이트코드를 한 라인 씩 실행
- ✓ 변경없이 재실행 시 바이트코드로 빠르게 실행



IDLE 개발환경

Python
PSF
Docs
PyPI
Jobs
Community

python™

Donate
GO
Socialize

About
Downloads
Documentation
Community
Success Stories
News
Events

```

# Python 3: List comprehensions
>>> fruits = ['Banana', 'Apple', 'Lime']
>>> loud_fruits = [fruit.upper() for fruit in fruits]
>>> print(loud_fruits)
['BANANA', 'APPLE', 'LIME']

# List and the enumerate function
>>> list(enumerate(fruits))
[(0, 'Banana'), (1, 'Apple'), (2, 'Lime')]

```

1
2
3
4
5

Compound Data Types

Lists (known as arrays in other languages) are one of the compound data types that Python understands. Lists can be indexed, sliced and manipulated with other built-in functions. [More about lists in Python 3](#)

Python is a programming language that lets you work quickly and integrate systems more effectively. >>> [Learn More](#)

Get Started

Whether you're new to programming or an experienced developer, it's easy to learn and use Python.

Download

Python source code and installers are available for download for all versions!

Latest: [Python 3.11.2](#)

Docs

Documentation for Python's standard library, along with tutorials and guides, are available online.

docs.python.org

Jobs

Looking for work or have a Python related position that you're trying to hire for? Our **relaunched community-run job board** is the



IDLE 개발환경

```
>>> import random
>>> print("Let's play rock-paper-scissors!")
...
Let's play rock-paper-scissors!
>>>
... # Ask the user to choose between rock, paper, or scissors
... user_choice = input("Enter your choice (rock/paper/scissors): ").lower()
...
Enter your choice (rock/paper/scissors): 1
>>>
... # Define the computer's choices
... computer_choices = ['rock', 'paper', 'scissors']
... computer_choice = random.choice(computer_choices)
...
SyntaxError: multiple statements found while compiling a single statement
>>> computer_choices = ['rock', 'paper', 'scissors']
...
>>> computer_choice = random.choice(computer_choices)
>>> if user_choice == computer_choice:
...     result = "Tie!"
... elif (user_choice == "rock" and computer_choice == "scissors") or \
...       (user_choice == "paper" and computer_choice == "rock") or \
...       (user_choice == "scissors" and computer_choice == "paper"):
...     result = "You win!"
... else:
...     result = "Computer wins!"
...

```

Ln: 31 Col: 4

Python. Latest: Python 3.11.2 docs.python.org community-run job board is the

IDLE 개발환경

■ 파이썬 인터프리터에 기본으로 포함된 파이썬의 통합 개발 환경(IDE)

- ✓ 파이썬과 Tkinter GUI 툴킷으로 개발
- ✓ 구문 강조, 자동 완성, 스마트 들여쓰기 등이 포함된 단순한 IDE 지향
- ✓ stepping, breakpoint, call stack을 확인할 수 있는 통합 디버거 환경 제공
- ✓ 파이썬 공식 홈페이지에서 다운로드 가능
 - <http://www.python.org>

주피터 노트북


jupyter
 Untitled Last Checkpoint: 2분 전 (unsaved changes)
 
 Logout

File Edit View Insert Cell Kernel Widgets Help
 Trusted Python 3 (ipykernel)












Code

```

In [1]: import random

사용자 입력을 받습니다.

In [4]: # Ask the user to choose between rock, paper, or scissors
user_choice = input("Enter your choice (rock/paper/scissors): ").lower()

# Define the computer's choices
computer_choices = ['rock', 'paper', 'scissors']
computer_choice = random.choice(computer_choices)

Enter your choice (rock/paper/scissors): 1

승패를 판단합니다.

In [ ]: # Determine the winner
if user_choice == computer_choice:
    result = "Tie!"
elif (user_choice == "rock" and computer_choice == "scissors") or \
      (user_choice == "paper" and computer_choice == "rock") or \
      (user_choice == "scissors" and computer_choice == "paper"):
    result = "You win!"
else:
    result = "Computer wins!"
    
```

In []:

주피터 노트북(Jupyter Notebook)

■ 오픈소스 기반의 웹 플랫폼

- ✓ <https://jupyter.org/>
- ✓ 파이썬을 비롯한 40여개의 프로그래밍 언어 지원
- ✓ 전통적인 소스코드-컴파일-실행 방식에서 벗어나 웹 기반 대화형 개발 및 실행 환경
- ✓ 문서화하여 다른 사람과 공유하기가 편리
- ✓ 마크다운(Markdown)을 이용하여 코드 관련 타이틀, 설명 등 작성 가능

구글 Colab(Google Colaboratory)



첨단공학부 파이썬 5강 (학습용).ipynb ☆

파일 수정 보기 삽입 런타임 도구 도움말



공유



명령어

+ 코드

+ 텍스트

모두 실행

연결



[]



#반지름값 사용자 입력

[]

#넓이 출력

```
print("넓이는", radius * radius * 3.141592, "입니다.")
```

[]

#둘레 출력

```
print("둘레는", 2 * radius * 3.141592, "입니다.")
```

변수

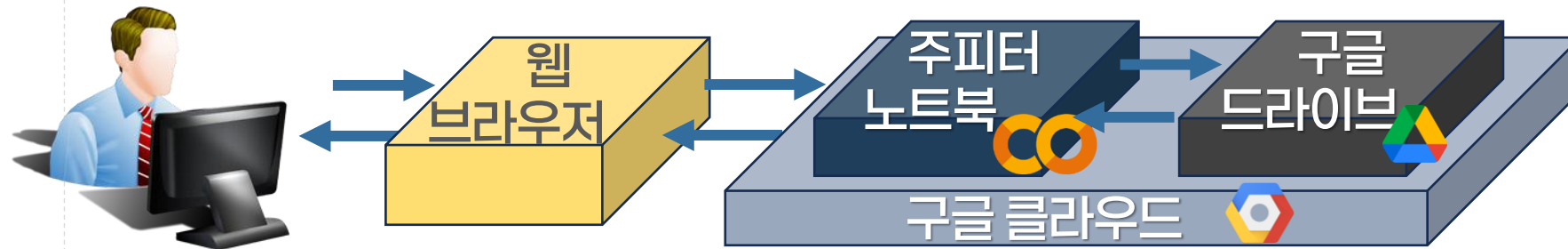
터미널



구글 Colab(Google Colaboratory)

- 2017년 과학 연구와 교육을 목적으로 개발
- 클라우드 기반 파이썬 개발 환경

- ✓ 주피터 노트북 + 구글 드라이브를 결합한 서비스
- ✓ 노트북 파일을 공유하여 협업 코딩 가능



- 고성능 컴퓨팅 리소스 제공

- ✓ CPU, GPU, TPU와 같은 고성능 연산 장비 활용 가능

학습하기

3

파이썬 기초 문법

데이터 타입

■ 데이터가 어떤 종류의 값을 가지는지를 구분

- ✓ 값의 성질과 연산 방식을 정의

■ 숫자 데이터 타입

- ✓ 정수(integer): 소수점이 없는 숫자
- ✓ 실수(floating point): 소수점이 포함되는 숫자

■ 문자 데이터 타입

- ✓ 유니코드(unicode) 기반의 문자열(string)
- ✓ 인용 부호 " 또는 '를 사용하여 표현

530000000

3.141592

'Hello World!'

"3.141592"

에코(echo)

- 입력으로 들어온 값을 아무 변화 없이 그대로 출력해 주는 기능

"Hello World, Python!"

"Hello World, Python!"

$123 * 456 / 789$

71.08745247148289

"Hello World, Python!"

$123 * 456 / 789$

71.08745247148289

함수(function)

■ 사전에 정의된 특정 작업을 수행하는 명령문의 집합

- ✓ 함수의 이름만으로 호출하여 실행할 수 있는 단위
- ✓ 함수의 예
 - print(): 화면에 데이터를 출력하는 작업
 - input(): 사용자 입력을 통해 데이터를 저장

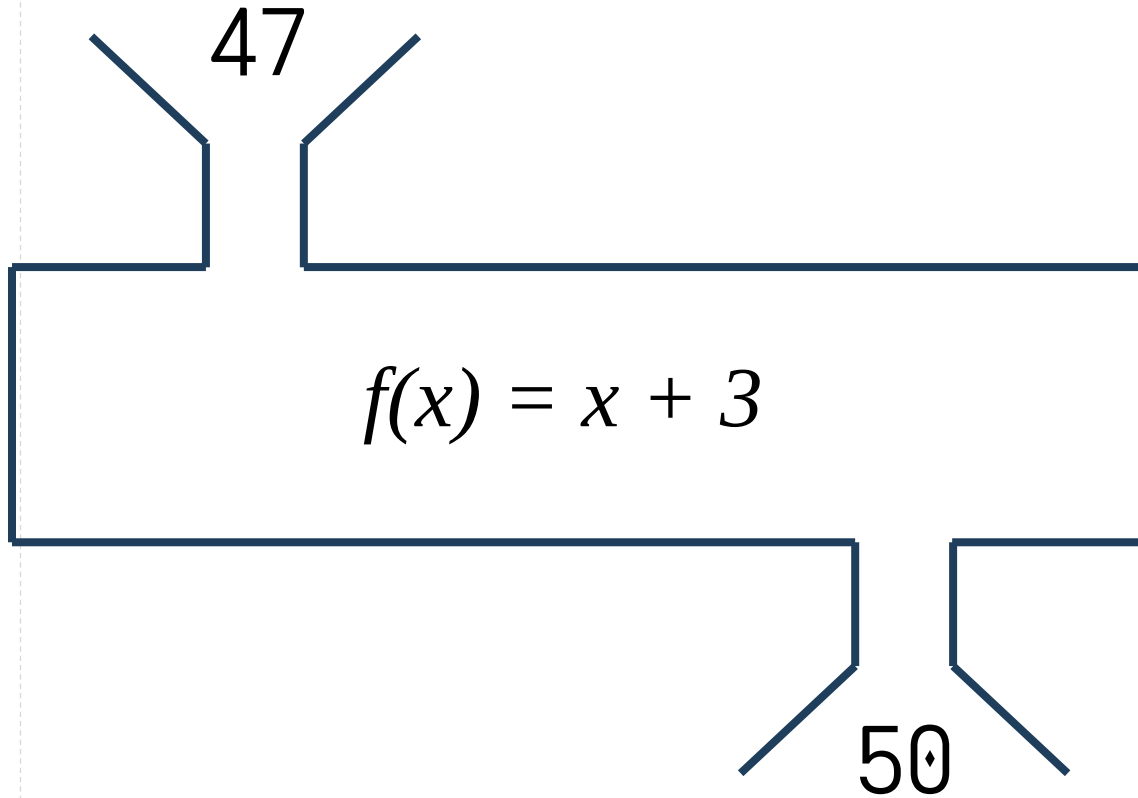
■ 함수 사용 형식

`print("Hello World!")`

함수의 이름

입력값(파라미터)

함수의 실행



함수의 실행

"Hello World!"

print(x): 화면에 x 출력

화면에 "Hello World!"를 출력

수학식 계산

- 파이썬 인터프리터는 숫자(int, float 등)에 대해 수학적 계산(덧셈, 뺄셈, 곱셈, 나눗셈)을 자동적으로 수행

```
print(25 + 7)
```

```
print(365 - 31)
```

```
print(52 * 4)
```

```
print(5 / 2)
```

- 괄호를 사용하여 여러 연산자의 연산 순서를 결정

$$\frac{10.5 + 2 \times 3}{45 - 3.5}$$

수학식 계산

- 파이썬 인터프리터는 숫자(int, float 등)에 대해 수학적 계산(덧셈, 뺄셈, 곱셈, 나눗셈)을 자동적으로 수행

```
print(25 + 7)
```

```
print(365 - 31)
```

```
print(52 * 4)
```

```
print(5 / 2)
```

- ✓ 괄호를 사용하여 여러 연산자의 연산 순서를 결정

```
print((10.5 + 2 * 3) / (45 - 3.5))
```

I 들여쓰기

■ 파이썬은 들여쓰기에 의존적 언어

- ✓ 타 프로그래밍 언어에서는 가독성 향상 목적
- ✓ 파이썬에서 코드의 논리적 집합인 블록을 표현

```
1 print("파이썬에 오신것을 환영합니다.")  
2 print("파이썬은 재미있습니다.")  
3     print("파이썬은 실용적입니다.")
```

■ 들여쓰기는 스페이스 4칸을 권장(PEP 8)

■ 블록 중첩 시 추가적인 4칸 들여쓰기 삽입

프로그래밍 스타일과 문서화

■ 적절한 공백의 사용

- ✓ 가독성 증대

```
print(3+4*4)
```

```
print(3 + 4 * 4)
```

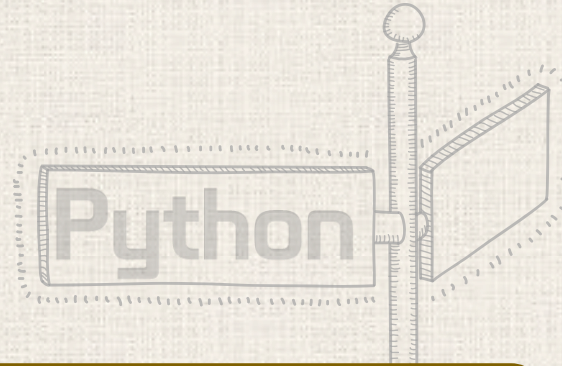
■ 주석(comment) 사용

- ✓ 프로그램의 실행에는 영향을 주지 않음
- ✓ 코드의 의미를 설명하기 위해 작성하는 코드의 한 요소

```
1 # 성적 계산
```

```
2 print((10.5 + 2 * 3) / (45 - 3.5))
```

다음시간에는



2강. 파이썬 시작!

➡ 3강. 변수와 연산자

4강. 순차 구조